

Project Title

AI Resume Analyzer Platform

Submitted By

ID	Name
23103380	Ehetisum Sharif
23103388	Nishat Tasnim

Course No: CSC 469

Course Title: Software Engineering

Section: D

Semester: Summer 2026

Date of submission: 23/08/2026

Submitted To

Avijit Biswas

Lecturer

Department of CSE.



Department of Computer Science and Engineering

IUBAT School of Computer Science and Engineering (ISCSE)

IUBAT – International University of Business Agriculture and Technology

Table of Contents

Part A: Feasibility of User and Functional Requirements	1
1.1 Project Overview	1
1.2 Feasibility Assessment	1
1.3 Decision Rationale.....	2
Part B: Justifying the Logical Structure of the System.....	2
2.1 Architecture Diagram of the AI Scoring Module.....	2
2.2 Three Reasons Behind the Structure	3
2.3 User and Business Needs.....	4
Conclusion	4
References.....	4
Appendix A: Process Model	5
Appendix B: Scheduling and Product Backlog	5
Appendix C: Risk Analysis Log	8

Part A: Feasibility of User and Functional Requirements

1.1 Project Overview

The AI Resume Analyzer Platform helps a job seeker improve a resume before applying. The user uploads a PDF or DOCX file of up to 5 MB and pastes the target job description. The system extracts the text, sends it with the job description to an AI model, and returns an ATS match score, feedback grouped into skills, content quality, structure and writing style, and the keywords missing from the resume.

The target users are students and fresh graduates who apply online and get no feedback when they are rejected. The second user type is the administrator, who manages accounts and watches system statistics. The backend is an ASP.NET Core 8 Web API in C#, the database is SQL Server through Entity Framework Core, the frontend is React 19, and the AI model is Llama 3 running locally through Ollama.

1.2 Feasibility Assessment

Before fixing the requirements we tested them in four ways. The two that changed the project most are explained below the table.

Type	What we checked	Result and decision
Technical	Can a resume be read and scored by an AI model inside our time target?	Yes for text based PDF and DOCX. Scanned files need OCR, so they were left out.
Operational	Will students use it, and can the team run it without a support desk?	Yes. The task is already done by hand today, and it needs only a browser and one server.
Economic	Can the project be finished with no budget?	Yes. Llama 3 runs locally, so there is no per request billing, and the other tools are free or student licensed.
Schedule	Can all the planned modules be built in one semester by three members?	Only after cutting scope. Job recommendation and mock interview were moved to a future release.

Technical feasibility: the least predictable requirement

The scoring engine was the most uncertain requirement, so we built a prototype that sent one resume and one job description to Llama 3 and printed the reply. The model produced useful feedback, but it

did not always return clean JSON and it slowed down when the machine was busy. Testing extraction on real resumes also showed that a scanned PDF returns no text at all.

Schedule feasibility: the reason scope was cut

We broke the plan into epics and tasks on a Scrum board and compared the total with the weeks available for three members. The first list did not fit, which forced us to separate the core system from the extra ideas. The result is the six sprint schedule in Appendix B.

1.3 Decision Rationale

The feasibility results changed the project in four concrete ways.

- A cloud AI service was not economically feasible, so the system was built around a local model. This later became a privacy advantage too, since no resume leaves our machine.
- Because the model does not always return clean JSON, the requirement was rewritten so a failed analysis shows an honest error instead of a score. A guard in the scoring service now cleans the reply and falls back safely.
- Since OCR was not feasible in the time left, accepted files were limited to text based PDF and DOCX, with a clear error when no text can be extracted.
- Because the full story list did not fit the semester, job recommendation and mock interview were deferred, keeping five modules complete instead of seven half finished.

This is why feasibility analysis matters. Each decision changed a requirement while changing it was still cheap. Found in the last sprint instead, the same problems would have meant deleting finished work or shipping something that fails in front of a user.

Part B: Justifying the Logical Structure of the System

The system uses a layered architecture with a separate client. The React frontend calls the Web API over HTTPS with a JWT token. Inside the API, controllers only receive requests, services hold the business logic, and Entity Framework Core is the only part that touches the database. The AI model sits outside the application, reached through a connector.

2.1 Architecture Diagram of the AI Scoring Module

The diagram below shows the AI scoring module. The two boxes with a red border belong to it: AiController and AiScoringService.

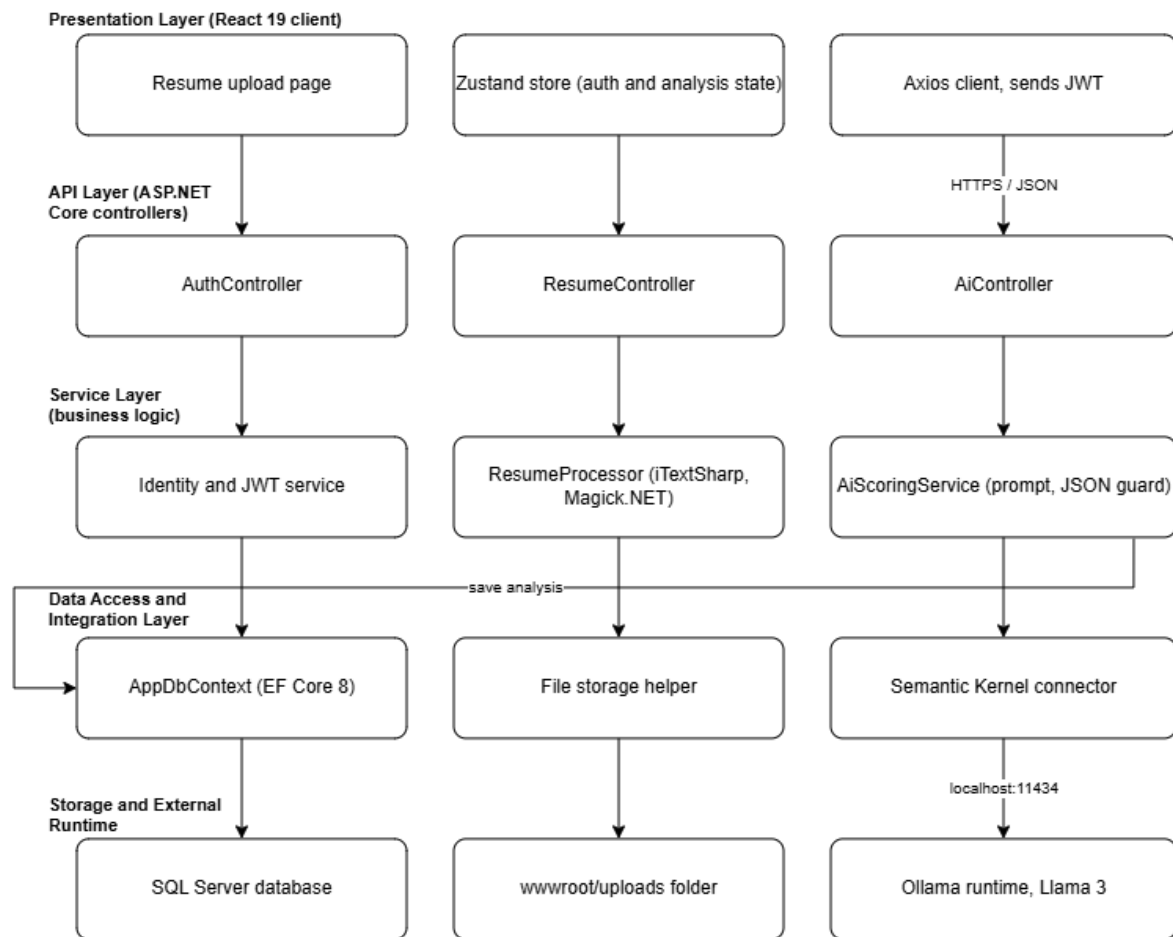


Figure 1: Layered architecture of the AI scoring module. The highlighted boxes form the module itself.

A request travels straight down the layers. AiController checks the token and passes the work to AiScoringService, which builds the prompt, calls Llama 3 through the connector and cleans the reply. The result is saved through AppDbContext, and the controller never touches the database or the model directly.

2.2 Three Reasons Behind the Structure

1. Separation of concerns keeps the AI part replaceable

All AI logic lives in one service behind one connector, so moving to another model or a hosted runtime changes only that service. This mattered because feasibility testing showed the model is the least stable part of the project.

2. A stateless API with tokens supports more than one client

The API keeps no session in memory. Every request carries a JWT token holding the identity and the role, so any server instance can answer it. A mobile client can be added later without rewriting the backend.

3. The layer boundary is also the security boundary

Because the frontend is separate, hiding a button in React protects nothing: anyone can call the API with Swagger or Postman. Placing the permission check at the controller layer enforces the rule at the only door into the system.

2.3 User and Business Needs

User needs shaped the structure as much as technical reasons. Users wanted a quick result, so extraction and preview happen once at upload and are stored, and the scoring request sends only prepared text. Users also wanted to see progress, so every analysis is saved as a row linked to its user and resume, and the dashboard simply reads those rows. On the business side the platform must run at no cost now but still be able to grow: because the model sits behind a connector, moving it to a stronger server later is a configuration change, not a rewrite.

Conclusion

Feasibility analysis and logical structure are connected here. Feasibility told us what was possible with no budget, one semester and three members, and it removed two features and two file types before any were built. The architecture then protected those decisions: the AI model is isolated behind a connector, the security check sits at the API boundary, and the data layer is the only path to the database. The result is a system small enough to finish and open to change where change is most likely.

References

- Microsoft. ASP.NET Core Documentation. Microsoft Learn. learn.microsoft.com/aspnet/core
- Microsoft. Semantic Kernel Documentation. learn.microsoft.com/semantic-kernel
- Meta AI (2024). Llama 3 Model Documentation. llama.meta.com
- Project repository: github.com/EhetisumSharif/ai-resume-analyzer

Appendix A: Process Model

We followed Scrum, an agile process model, with short sprints of about one week. Scrum was chosen over the waterfall model for one reason found during feasibility testing: the behaviour of the AI model could not be predicted from a document, only from running it. A process that allowed the requirements to change after each sprint was therefore safer than a process that fixed everything at the start.

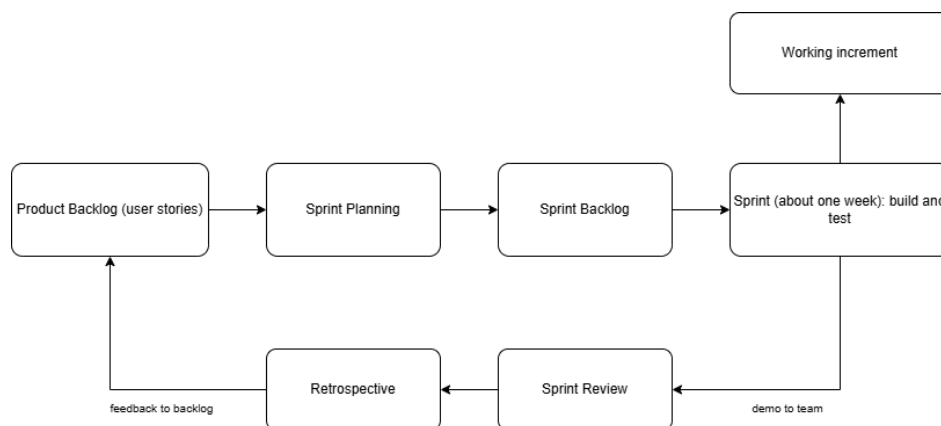


Figure 2: The Scrum process model used in this project.

Each sprint began with planning, where tasks were moved from the product backlog into the sprint backlog. At the end of the sprint the working increment was demonstrated to the team, and the retrospective sent lessons and unfinished work back into the product backlog. Work was divided by layer so that the three members could work in parallel: Ehetisum Sharif took the C# API and the AI integration, Md. Sajid took the React frontend and state management, and Nishat Tasnim took the database and document processing. Appendix B shows how this split appears on the board.

Appendix B: Scheduling and Product Backlog

The work was planned on a Scrum board and divided into six sprints, each mapped to one epic. Table B.1 shows the schedule and Table B.2 lists every task on the board with its owner and its state on the day this report was written.

Table B.1: Sprint schedule

Sprint	Dates (2026)	Epic worked on	Tasks	Progress
Sprint 1	07/10 to 07/19	Project Setup & Auth	10	10 of 10 done
Sprint 2	07/19 to 07/26	Resume Upload & Processing	8	8 of 8 done
Sprint 3	07/28 to 08/04	AI Scoring & Feedback	7	7 of 7 done
Sprint 4	08/05 to 08/12	AI Scoring & Feedback + Dashboard and job description APIs	6	6 of 6 done
Sprint 5	08/20 to 08/23	Dashboard & UI Polish	6	5 of 6 done

Sprint 6	08/25 to 08/28	Integration QA & Demo Prep	6	1 of 6 done
----------	----------------	----------------------------	---	-------------

Sprints 1 to 4 closed with every task finished. Sprint 5 has one task open, the end to end bug fixes, and Sprint 6 is the final QA and demonstration sprint, which starts after this report is submitted. Out of 43 tasks on the board, 37 are done and 6 are still open.

Table B.2: Product backlog, all tasks

Key	Task	Assignee	Sprint	Status
S-11	Setup ASP.NET Core backend project structure	Ehetisum Sharif	Sprint 1	Done
S-14	Setup React + Vite + TS project	Md. Sajid	Sprint 1	Done
S-15	Configure Tailwind CSS	Md. Sajid	Sprint 1	Done
S-16	Build login/signup UI pages	Md. Sajid	Sprint 1	Done
S-17	Setup Zustand store for auth state	Md. Sajid	Sprint 1	Done
S-12	Implement ASP.NET Identity (signup/login)	Ehetisum Sharif	Sprint 1	Done
S-13	Implement JWT token generation & validation	Ehetisum Sharif	Sprint 1	Done
S-18	Setup SQL Server + EF Core connection	Nishat Tasnim	Sprint 1	Done
S-19	Design DB schema (User Resume Analysis JobDescription)	Nishat Tasnim	Sprint 1	Done
S-20	Create initial EF migrations	Nishat Tasnim	Sprint 1	Done
S-21	Build upload API endpoint	Ehetisum Sharif	Sprint 2	Done
S-22	Implement file validation (type/size)	Ehetisum Sharif	Sprint 2	Done
S-23	Build drag-and-drop upload UI	Md. Sajid	Sprint 2	Done
S-24	Build resume preview component	Md. Sajid	Sprint 2	Done
S-25	Wire upload UI to backend via Axios	Md. Sajid	Sprint 2	Done
S-26	Integrate iTextSharp for PDF text extraction	Nishat Tasnim	Sprint 2	Done
S-27	Integrate Magick.NET for preview image generation	Nishat Tasnim	Sprint 2	Done
S-28	Store extracted resume data in DB	Nishat Tasnim	Sprint 2	Done
S-29	Integrate Semantic Kernel with backend	Ehetisum Sharif	Sprint 3	Done
S-30	Design AI prompt for resume vs job description scoring	Ehetisum Sharif	Sprint 3	Done

S-31	Implement ATS score generation (0-100) endpoint	Ehetisum Sharif	Sprint 3	Done
S-32	Implement analysis result storage in DB	Nishat Tasnim	Sprint 3	Done
S-33	Implement AI timeout (15s) + retry logic	Nishat Tasnim	Sprint 3	Done
S-34	Build result loading state UI	Md. Sajid	Sprint 3	Done
S-35	Build score display card component	Md. Sajid	Sprint 3	Done
S-37	Implement missing-skill/keyword detection	Ehetisum Sharif	Sprint 4	Done
S-38	Build feedback UI (4-category sections)	Md. Sajid	Sprint 4	Done
S-36	Implement 4-category feedback logic (Skills Content Structure Style)	Ehetisum Sharif	Sprint 4	Done
S-39	Build missing-skills highlight component	Md. Sajid	Sprint 4	Done
S-40	Build dashboard backend API (fetch past resumes/scores)	Nishat Tasnim	Sprint 4	Done
S-41	Build job description storage/retrieval endpoint	Nishat Tasnim	Sprint 4	Done
S-42	Build personal dashboard UI	Md. Sajid	Sprint 5	Done
S-43	Build side-by-side review screen	Md. Sajid	Sprint 5	Done
S-44	Build progress tracking chart (score over time)	Md. Sajid	Sprint 5	Done
S-45	Handle multiple resume version storage/comparison logic	Nishat Tasnim	Sprint 5	Done
S-46	DB cleanup & indexing for performance	Nishat Tasnim	Sprint 5	Done
S-47	End-to-end flow bug fixes (frontend/backend/AI)	Ehetisum Sharif	Sprint 5	To Do
S-48	End-to-end testing (upload -> score -> feedback flow)	Ehetisum Sharif	Sprint 6	To Do
S-49	Responsive design fixes (mobile/tablet/desktop)	Md. Sajid	Sprint 6	Done
S-50	Error handling QA (invalid file AI timeout network errors)	Nishat Tasnim	Sprint 6	To Do
S-51	Final bug bash	Ehetisum Sharif	Sprint 6	To Do
S-52	Prepare demo script	Md. Sajid	Sprint 6	To Do
S-53	Finalize report & presentation slides	Nishat Tasnim	Sprint 6	To Do

Two features that appeared in the first plan, job recommendation and mock interview, were never added to the board. They were removed during the schedule feasibility check described in Part A, which is why the board contains only the five core modules.

Appendix C: Risk Analysis Log

The risk log was reviewed at the start of every sprint. Likelihood and impact are recorded as high, medium or low, and the last column records the action that was actually taken rather than a general plan.

ID	Risk	Likelihood	Impact	Response taken
R-01	The AI model returns text that is not valid JSON and breaks the result page	High	High	The reply is cut between the first and last brace and passed through a guard. If parsing still fails, a safe result with a clear message is returned.
R-02	AI analysis takes longer than the 15 second target on a loaded machine	Medium	Medium	A smaller Llama 3 model was chosen and one prompt now returns every part of the result instead of several calls.
R-03	Text extraction fails for scanned or two column resumes	Medium	Medium	Only text based PDF and DOCX are accepted, and a clear error is shown when extraction returns nothing.
R-04	Access control left switched off after development testing	Medium	High	Every controller is reviewed before release, and a feature is not treated as finished until its access check is enabled.
R-05	Scope too large for one semester with three members	High	Medium	Job recommendation and mock interview stories were moved to the backlog after the schedule feasibility check.

R-06	Team member unavailable during a sprint	Medium	Medium	Work is divided by layer, and the backlog is kept in one shared place so any member can pick up a story.
------	---	--------	--------	--

The two risks with the highest combined score, R-01 and R-04, both produced permanent changes in the system: the JSON guard in the scoring service, and the rule that a feature is not finished until its access check is enabled and tested.